

MIDAS: Load-Balanced Metadata-Outsourcing ID-Embedded Data Auditing Scheme for Mobile Devices

Meiqin Tang¹, Jian Shen¹, Sheng Wen¹, Tianqing Zhu¹, *Member, IEEE*, and Yang Xiang², *Fellow, IEEE*

Abstract—With the rapid growth of mobile applications, large volumes of private data are continuously generated on mobile devices and increasingly outsourced to cloud storage services. Due to the limited computational and energy resources of mobile devices, enabling efficient and secure periodic auditing of cloud-stored data remains a challenging task. Existing cloud data auditing schemes, such as Provable Data Possession and Proof of Retrievability, require computation-intensive operations that are impractical for resource-constrained mobile environments. To address this challenge, we propose MIDAS, a mobile-friendly data auditing scheme that supports secure outsourcing of auditing metadata with balanced resource utilization. MIDAS adopts a two-phase auditing metadata structure, in which computation-intensive operations are offloaded to edge nodes, while mobile devices only perform lightweight processing. In addition, identity information of edge nodes is embedded into the auditing metadata to support traceability and accountability. To further improve scalability, we design a resource scheduling model for Multi-MU and Multi-EN scenarios and propose the proportional load-balanced scheduling algorithm to achieve efficient and load-balanced metadata generation. The security of MIDAS is rigorously proven by reducing it to the discrete logarithm problem in the RSA quadratic residue subgroup within the random oracle model. Experimental results in a real-world environment demonstrate that MIDAS significantly reduces computational overhead on mobile devices, maintains acceptable communication cost, and achieves superior scalability compared with existing schemes.

Index Terms—Cloud data auditing, metadata outsourcing, resource scheduling, identity management, mobile edge computing.

I. INTRODUCTION

RECENT statistics indicate that mobile devices now contribute approximately 60%–65% of global web traffic,

reflecting a significant shift toward mobile-first data generation and consumption [1], [2], [3]. This rapid increase is primarily driven by the proliferation of high-performance mobile devices and the widespread use of data-intensive applications such as high-resolution photography, video streaming, online gaming, and social networking, which continuously produce massive volumes of multimedia content. For mobile devices, such an enormous amount of personal data has already surpassed their local storage capacity. Consequently, the use of cloud storage services, such as iCloud, Google Drive, and Huawei Cloud, has become a common practice, providing users with a more convenient and scalable way to store and manage their data [4]. However, recent advances in software testing techniques, such as large-scale fuzzing, have exposed a growing number of software vulnerabilities [5], [6], [7], [8], while the rapid development of AI-driven attack techniques further amplifies the scale and sophistication of cyber threats [9], [10], [11], [12]. As a result, data stored and processed in cloud environments is facing increasingly severe integrity and security risks. Therefore, cloud environments require periodic integrity auditing to ensure data availability and reliability.

Numerous cloud data auditing schemes have been proposed in recent years, which can generally be classified into two categories: Provable Data Possession (PDP) schemes [13], [14] and Proof of Retrievability (POR) schemes [15], [16]. In PDP-based auditing protocols, the client first generates a set of homomorphic verification tags for the data before outsourcing it to the cloud [17], [18], [19]. During the auditing phase, the cloud server uses these tags together with the stored data blocks to produce a proof of integrity in response to the auditor's challenge, and the client verifies this proof to ensure that the outsourced data remains intact. In contrast, POR-based schemes employ erasure coding to introduce redundancy into the data and insert a number of sentinels, special predetermined blocks, at random positions [20], [21], [22]. During an audit, a subset of sentinels is randomly selected and checked for modification, enabling probabilistic assurance of data integrity. Furthermore, the embedded erasure code allows the recovery of a small portion of lost or corrupted data.

However, neither PDP- nor POR-based auditing protocols can be directly deployed on mobile devices. The generation of homomorphic verification tags, erasure codes, or sentinels requires intensive computation, which is infeasible for mobile devices with limited processing power and battery capacity [23],

Received 24 December 2025; revised 29 January 2026; accepted 28 February 2026. Date of publication 6 March 2026; date of current version 12 May 2026. This work was supported in part by the National Key R&D Program of China under Grant 2023YFB2703700, in part by the National Natural Science Foundation of China under Grant U21A20465, Grant 62302457, and Grant 62172292, in part by the Fundamental Research Funds of Zhejiang Sci-Tech University under Grant 22222266-Y, and in part by the Program of Zhejiang Key Laboratory of Digital Fashion and Data Governance under Grant 2024E10049. (Corresponding author: Jian Shen.)

Meiqin Tang and Jian Shen are with the School of Information Science and Engineering, Zhejiang Sci-Tech University, Hangzhou, Zhejiang 310018, China (e-mail: tmq220@163.com; s_shenjian@126.com).

Sheng Wen and Yang Xiang are with the School of Software and Electrical Engineering, Swinburne University of Technology, Hawthorn, VIC 3122, Australia (e-mail: swen@swin.edu.au; yxiang@swin.edu.au).

Tianqing Zhu is with the Faculty of Data Science, City University of Macau, Macau 999078, China (e-mail: tqzhu@cityu.edu.mo).

Digital Object Identifier 10.1109/TDSC.2026.3670871

[24]. For instance, generating homomorphic verification tags for a 4 GB file may take about two hours on a standard desktop machine, whereas a mobile device would require significantly more time under full CPU load. Such heavy computational and energy demands make existing cloud auditing schemes impractical for mobile environments [25], [26].

In recent years, several studies have integrated cloud data auditing with the Mobile Edge Computing (MEC) paradigm to offload computation-intensive auditing tasks from mobile devices to nearby edge nodes [27], [28]. By offloading computation to edge servers with ample processing power, these schemes reduce the resource burden on mobile devices. However, most existing MEC-based auditing schemes assume that edge nodes are fully trusted and will faithfully execute the delegated operations. In practice, this assumption may not hold, as edge servers can be compromised or become semi-trusted due to their distributed and less-protected deployment environments [29], [30]. Once a malicious or faulty edge node tampers with auditing metadata, the integrity of the outsourced data can no longer be guaranteed.

To overcome this limitation, we propose MIDAS, a load-balanced metadata-outsourcing ID-embedded data auditing scheme that enables secure metadata generation and verification even under untrusted edge nodes. By employing a two-phase ID-embedded metadata structure, MIDAS achieves both verifiable security and lightweight computation in mobile edge environments. The main contributions are as follows:

- We propose a two-phase auditing metadata structure integrated with an ID-embedded mechanism. In this design, the edge computing node pre-generates verification tags and auxiliary parameters, while the mobile device produces the final auditing metadata based on the data to be uploaded and the received tags. All modular exponentiation operations involved in metadata generation are performed at the edge node, whereas only the mobile device that holds both the private key and the data can produce valid auditing metadata. This design achieves secure metadata outsourcing by preventing untrusted edge nodes from forging or manipulating the auditing information. Furthermore, the metadata structure incorporates the device identity (ID) as a built-in attribute, enabling lightweight authentication and simplified key management.
- We construct a security-aware computing resource scheduling model for multi-edge-multi-device environments. In this model, the outsourced metadata generation task of a single mobile device can either be executed collaboratively by multiple edge nodes in parallel, or handled by a powerful edge node that serves multiple mobile devices simultaneously. To achieve balanced workload distribution and efficient resource utilization, we establish a scheduling mechanism that takes into account factors such as the security coefficient, file size, and data block granularity. This design maximizes the computing efficiency of edge resources while ensuring the secure and reliable execution of metadata outsourcing tasks.
- We provide comprehensive security and performance evaluations of MIDAS. First, the security of MIDAS is formally proven under the random oracle model by reducing it to the

discrete logarithm problem in the RSA quadratic residue subgroup. Then, we implement MIDAS in a realistic edge computing environment and evaluate its efficiency on an iPhone 15 Pro. Experimental results show that, compared with mainstream cloud data auditing schemes, MIDAS incurs the lowest computational overhead for the mobile user. Processing time of MIDAS for a 10 GB file with 4 KB data block is reduced to only about 14 minutes.

II. RELATED WORK

This section presents the related work on MIDAS, with one line of research focusing on optimizing the computational overhead of auditing metadata and another focusing on resource scheduling.

A. Auditing Metadata Generation Optimization

Li et al. and Wang et al. proposed two data auditing schemes based on different constructions of Chameleon Hash, both of which rely on a precomputation strategy [31], [32]. In these schemes, a set of Chameleon Hash values is generated in advance, and the mobile device only needs to compute auxiliary parameters that collide with the precomputed hashes. Although this design reduces online computation, it essentially trades storage for computation, making it unsuitable for mobile devices. Once the precomputed hash values are exhausted, users can no longer verify data integrity, and maintaining a large pool of precomputed hashes imposes substantial storage overhead on resource-constrained mobile devices.

In contrast, Yang et al. introduced a decentralized self-auditing scheme for multi-cloud storage, in which auditing metadata generation is avoided by directly encoding the original data. [33]. Since the encoding process involves only arithmetic operations, the computational cost is relatively low. However, this approach introduces strict limitations on data block size. With a security parameter of 1024 bits, the block size is restricted to 160 bits, leading to a significant data expansion, typically by a factor of two to three after encoding, which severely limits its practicality for mobile environments.

Different from the above two categories, another line of research focuses on auditing metadata ownership transfer. Yuan et al. presented a data auditing protocol for group data sharing that enables dynamic data operations via tag conversion. [34]. Building on this work, Wang et al. introduced a threshold ownership transfer for cloud EMRs Auditing [35]. Shen et al. addressed tag conversion in data ownership transfer by requiring users to provide auxiliary information for regenerating tags under new ownership [36], while Rabaninejad et al. proposed a proxy re-signature-based tag conversion scheme [37]. Despite their differences, all these schemes require users to generate audit tags for all data blocks prior to ownership transfer or verification. This requirement renders them impractical for resource-constrained mobile devices, where large-scale tag generation is prohibitively expensive.

The final line of work follows a similar intuition to MIDAS by utilizing third-party computing nodes to assist mobile devices in auditing metadata generation. Wang et al. proposed two

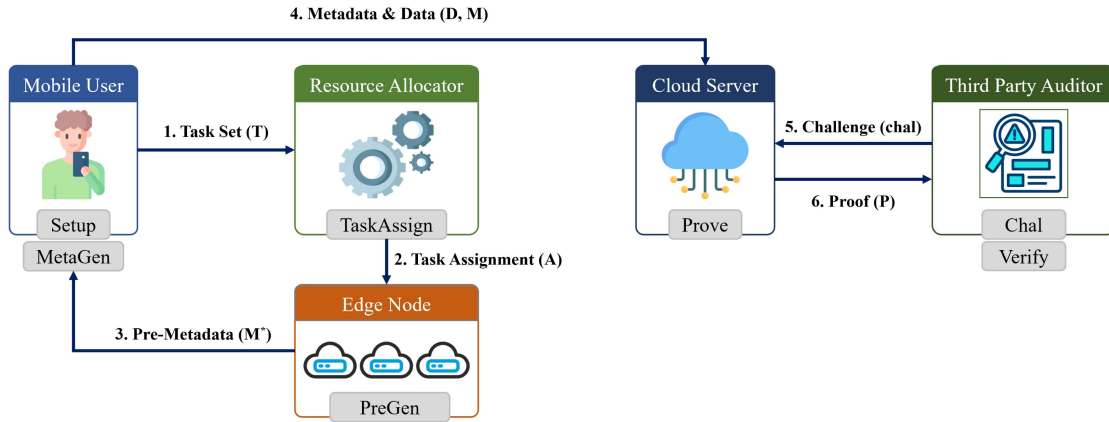


Fig. 1. Overview of MIDAS.

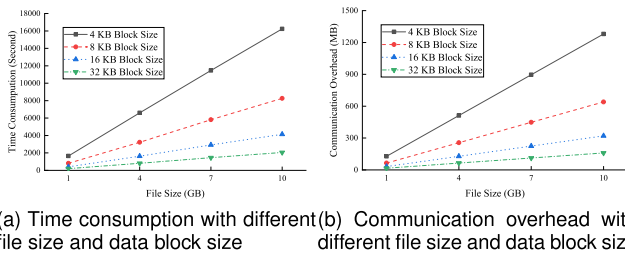


Fig. 2. Performance analysis of EN for PreGen.

data auditing schemes based on this model [38], [39]. In the former, users generate auditing tags locally and authorize trusted third-party nodes to conduct auditing through certificates. The latter adopts an ID-based cryptographic framework to simplify key management, and delegates auditing tag generation to authorized third-party nodes, thereby reducing the computational burden on users. In a different application context, Lu et al. designed a data auditing scheme for smart meter data in smart grids [40], where a trusted auditing node is deployed near the meters to handle computation-intensive operations. This approach is specifically tailored to smart grid environments. As a result, these schemes introduce strong trust assumptions on the authorized third-party node and do not offer security guarantees against its potential misbehavior, which fundamentally limits their applicability in untrusted or heterogeneous environments.

Overall, existing data auditing schemes that aim to reduce auditing metadata overhead suffer from inherent limitations when applied to resource-constrained mobile environments. Approaches based on precomputation or data encoding either impose excessive storage overhead or introduce significant data expansion, while metadata ownership transfer schemes still require users to generate audit tags for all data blocks in advance. Although several studies leverage third-party computing nodes to assist mobile devices, they typically rely on strong trust assumptions and lack effective mechanisms to prevent or hold accountable potential misbehavior of the authorized nodes. Consequently, none of the existing solutions simultaneously support secure auditing metadata outsourcing, low client-side overhead, and accountability in heterogeneous and potentially untrusted environments. These limitations motivate the design of MIDAS,

which aims to enable secure, accountable, and load-balanced auditing metadata outsourcing tailored for resource-constrained mobile users.

B. Resource Scheduling for Cloud Computing

Zhao et al. addressed the challenges of diverse and rapidly growing resource demands as well as insufficient scheduling efficiency in cloud-edge collaborative intelligent transportation systems, and proposed a traffic resource scheduling model based on virtualized digital twins [41]. The proposed approach constructs a global resource view through virtual-physical fusion mapping, a cloud-edge collaborative multi-resource scheduling architecture, and a unified virtualized resource representation. Furthermore, a reinforcement learning model based on node-importance strategies is designed to learn critical nodes and their relationships, enabling on-demand adaptive scheduling.

Sedighi et al. investigated the problem of heterogeneous GPU task arrival times and dynamically varying resource demands in cloud environments, and proposed a dynamic task scheduling and adaptive GPU resource allocation framework [42]. Their solution adopts a hybrid software-hardware GPU partitioning strategy together with a dynamic scheduling mechanism that supports repartitioning, reallocation, and task eviction, thereby improving resource utilization and reducing task completion time. The effectiveness of the framework is validated through simulations driven by real production traces.

Liu et al. focused on the scheduling challenges of instance-intensive workflows under resource-constrained conditions in containerized hybrid clouds, and proposed a batch scheduling scheme for multi-mode tasks based on batch scheduling strategies [43]. The proposed method prioritizes workflows with high repetition probability and records historically selected virtual machines and execution modes, thereby reducing data transmission overhead and computational randomness. Combined with improved heuristic and hybrid algorithms to solve the multi-mode scheduling model, the approach is able to reduce scheduling overhead even as the workflow scale increases.

Cao et al. addressed the problem of severe resource contention caused by concurrent multi-task execution and service quality fluctuations under deadline constraints in edge computing environments, and proposed a resource-aware task grouping scheduling strategy [44]. The proposed solution constructs a grouped and shared-state edge resource management framework and formulates deadline-constrained multi-task scheduling with resource contention. At the strategy level, it integrates task regrouping and priority response models, a resource-aware greedy scheduling algorithm, and task adjustment mechanisms, significantly improving deadline satisfaction rates and resource utilization.

Zhu et al. investigated the limitations of container cloud task scheduling in real-world systems caused by sparse samples and features, as well as the difficulty of capturing complex task–resource relationships in state representations, and proposed a two-stage semantic-aware container task scheduling framework [45]. In the first stage, large-scale offline cluster traces are used to construct aligned multimodal data, and cross-modal contrastive learning is introduced for pretraining to learn semantic representations of task–resource affinity. In the second stage, reinforcement learning is employed for online fine-tuning based on runtime feedback, mapping state representations to scheduling actions, thereby improving convergence efficiency and resource utilization.

Although the above studies have achieved substantial progress in cloud resource scheduling, they mainly focus on general cloud or edge scheduling objectives, such as completion time, cost efficiency, fairness, deadline satisfaction, resource utilization, or semantic representation learning. However, data auditing metadata generation is rarely modeled as a core workload. Specifically, the computational overhead of auditing metadata generation depends not only on data scale but also heavily on security parameter configurations and data block sizes. Such security-aware and data-dependent computational characteristics have not been sufficiently captured in existing resource scheduling models, making current approaches difficult to directly apply to auditing metadata generation scenarios.

III. RELATED MODELS

A. System Model

As illustrated in Fig. 1, MIDAS consists of five types of entities: the Mobile User (MU), Resource Allocator (RA), Edge Node (EN), Cloud Server (CS), and Third-Party Auditor (TPA).

The introduction of edge nodes and the separation of metadata generation from the auditing phase are motivated by practical system characteristics and performance requirements. Traditional outsourced auditing schemes typically rely on a centralized authorized node to perform cryptographic computations, which may lead to high overhead and performance bottlenecks in scenarios with large user populations or high network latency. By offloading computation-intensive metadata generation to nearby edge nodes, the proposed design reduces the computational burden on resource-constrained mobile users and enables block-level parallelism, thereby improving scalability in multi-user and large-scale data scenarios. In addition, metadata generation is typically a one-time or low-frequency operation, whereas

auditing is performed periodically. Separating these two phases significantly reduces the online auditing overhead and makes the scheme more suitable for long-term storage with frequent integrity verification.

The detailed descriptions of these five entities are as follows:

- **MU:** The MUs are resource-constrained devices that outsource their data to the CS to alleviate local storage pressure. During the metadata generation phase, each MU collaborates with ENs by providing necessary information and locally aggregating the received pre-metadata. During the auditing phase, the MU receives and reviews the audit results reported by the TPA.
- **RA:** The RA is responsible for coordinating the system in multi-MU and multi-EN environments. Specifically, during the task assignment phase, the RA schedules and assigns block-level metadata generation tasks of different MUs to available ENs, aiming to achieve load-balanced and efficient execution.
- **EN:** The ENs assist MUs by performing computation-intensive pre-metadata generation tasks. Upon receiving assigned tasks from the RA, each EN independently generates pre-metadata for the designated data blocks and returns the results to the corresponding MU.
- **CS:** The CS provides large-scale storage services for MUs. It stores both the outsourced data and the corresponding auditing metadata, and during the auditing phase, it generates integrity proofs in response to audit challenges from the TPA.
- **TPA:** The TPA acts as an independent auditor. It periodically initiates integrity audits by generating audit challenges, verifies the proofs returned by the CS, and reports the audit results to the MUs.

To distinguish different stages, the intermediate results generated by ENs are referred to as pre-metadata, which serve only as inputs to the final metadata generation. The metadata generated by the MU for integrity auditing are referred to as auditing metadata and are abbreviated as metadata when no ambiguity arises.

Under this system model, the framework of MIDAS is summarized in Definition 1.

Definition 1. MIDAS consists of seven algorithms

- $Setup(\lambda) \rightarrow (SP, SK, PK)$: Given the security parameter λ , the MU computes the corresponding system parameters SP . In addition, all entities generate their respective public and private key pairs. Let SK denote the set of all private keys, which are kept locally, and PK denote the set of all public keys, which are publicly shared.
- $TaskAssign(T, R) \rightarrow (A)$: Given the current metadata generation task set T and the computational resources of edge nodes R , the RA computes the task assignment mapping A for metadata generation.
- $PreGen(t) \rightarrow (M^*)$: Given the assigned subtask t , the EN computes the pre-metadata M^* , which is then sent to the MU.
- $MetaGen(M^*, D = \{d_1, d_2, \dots, d_n\}) \rightarrow (M)$: Given the pre-metadata M^* and the data D with n data blocks to be uploaded by the MU, the MU generates the auditing metadata M . Finally, D and M are sent to the CS.

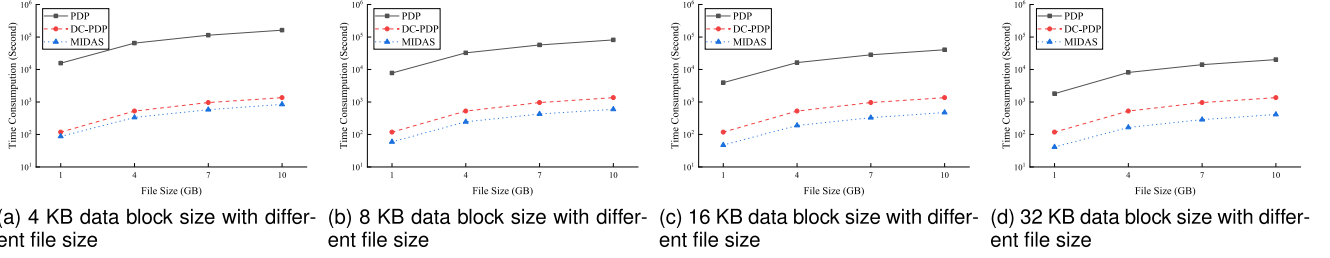


Fig. 3. Time consumption comparison of mobile user for MetaGen with different schemes (PDP, DC-PDP, MIDAS).

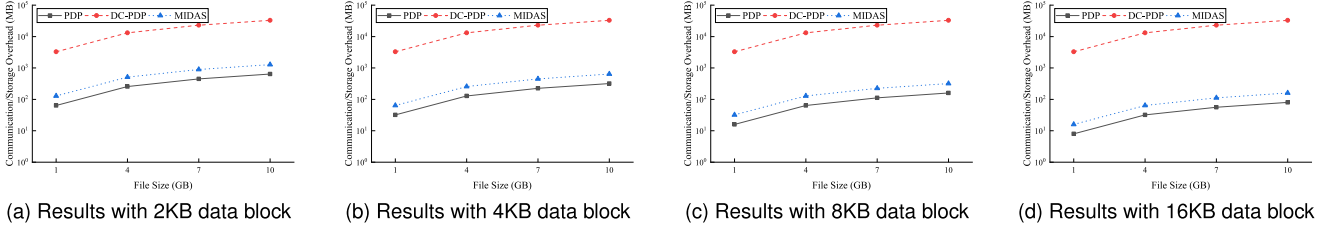


Fig. 4. Communication/Storage overhead comparison of mobile user for MetaGen with different schemes (PDP, DC-PDP, MIDAS).

- $Chal(|D|) \rightarrow (chal)$: Given the number of data blocks $|D|$, the TPA generates an auditing challenge $chal$, which is then sent to the CS.
- $Prove(chal, D, M) \rightarrow (P)$: Given the auditing challenge $chal$, the data D , and metadata M , the CS computes the integrity proof P , and sends it to the TPA.
- $Verify(chal, P) \rightarrow (T/F)$: Given the auditing challenge $chal$ and the integrity proof P , the TPA verifies the data integrity and outputs the verification result T/F , which is then sent to the MU.

System Workflow: Based on the seven algorithms defined in Definition 1, the overall workflow of MIDAS proceeds as follows. During the *Setup* phase, the MU initializes the system by generating the system parameters SP and distributing the corresponding public keys PK among all entities. Next, the RA executes the *TaskAssign* algorithm to schedule the metadata generation tasks of multiple mobile users across heterogeneous edge nodes, considering their available computational resources. Each assigned edge node then runs the *PreGen* algorithm to compute the pre-metadata M^* for the corresponding subtask and sends it to the MU.

After receiving M^* , the MU executes the *MetaGen* algorithm to generate the final auditing metadata M based on its local data D , and uploads both D and M to the CS. The TPA periodically initiates an auditing round by invoking the *Chal* algorithm to produce a random challenge $chal$, which is transmitted to the CS. Upon receiving $chal$, the CS executes the *Prove* algorithm to compute the integrity proof P for the corresponding data D and returns P to the TPA. Finally, the TPA performs the *Verify* algorithm using $chal$ and P to validate the integrity of the cloud-stored data, and outputs the verification result T/F to the MU.

Through this workflow, MIDAS enables secure and efficient metadata outsourcing, load-balanced computation across edge

nodes, and lightweight integrity auditing for mobile devices. Next, we analyze the potential threats faced by MIDAS and present the formal definition of its security model.

B. Threat Analysis and Security Model

From the MU's perspective, the RA and TPA are regarded as fully trusted entities, typically operated or endorsed by governmental or certified organizations. They have no incentive to deviate from the protocol. Importantly, the protocol does *not* require them to possess high-end storage or computational resources; only lightweight capability is sufficient to run the scheduling algorithm (for RA) and the challenge-verification logic (for TPA).

In contrast, the CS is considered semi-trusted. As a commercial service provider, the CS may behave dishonestly to maximize profit or maintain reputation. If data stored in the CS is lost or corrupted, it may attempt to generate a forged proof P to pass the TPA's verification and conceal the data loss. The ENs, on the other hand, are assumed to be untrusted and opportunistic. They may return incorrect pre-metadata M^* or perform incomplete computations to gain rewards without consuming their actual computational resources. However, an EN cannot forge the final auditing metadata M , since only the MU holds the private key required for metadata generation.

All entities communicate through secure and authenticated channels, and the MU is assumed to be fully honest, strictly following the protocol specifications.

Based on the above threat analysis, we now present the formal security model of MIDAS.

First, *Correctness*: if the pre-metadata m^* and the data D are correctly generated and remain intact, the integrity proof produced by the CS must always pass the verification of the TPA.

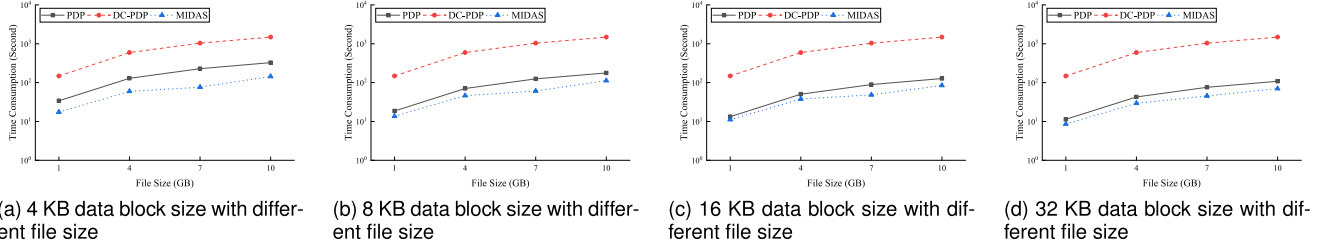


Fig. 5. Time consumption comparison of cloud server for Proof with different schemes (PDP, DC-PDP, MIDAS).

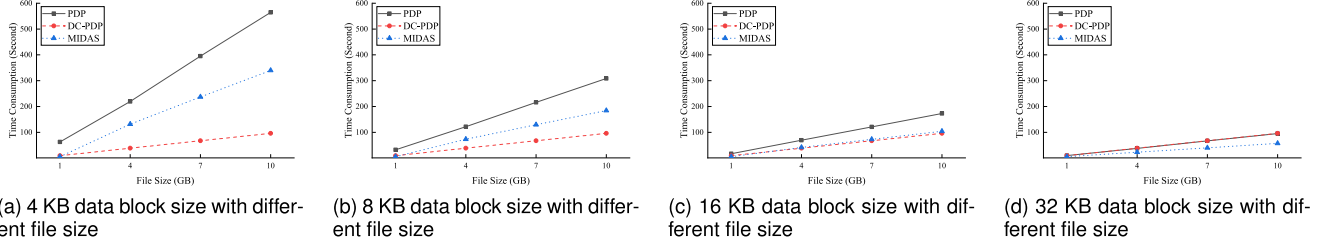


Fig. 6. Time consumption comparison of third-party auditor for Chal and Verify with different schemes (PDP, DC-PDP, MIDAS).

Definition 2 (Correctness): The correctness of MIDAS holds when all participants follow the protocol faithfully, that is, the integrity proof generated by the CS will always be accepted by the TPA.

Next, *Metadata-outsourced Data Possession*: if the integrity proof generated by the CS can pass the verification of the TPA, then the CS must have correctly stored all the data blocks involved in the challenge.

Definition 3 (Metadata-Outsourced Data Possession): Let MIDAS defined in Definition 1. MIDAS satisfies Metadata-Outsourced Data Possession if no PPT adversary $\mathcal{A}$ controlling the EN and CS can produce a valid proof for a challenge on blocks it does not possess, except with negligible probability. The following game $Game^{MODP}$ is executed between a simulator $\mathcal{B}$ (simulating MU and TPA) and the adversary $\mathcal{A}$. TaskAssign is omitted in $Game^{MODP}$ since resource scheduling is not related to the security. Let λ be the security parameter.

- **Setup:** $\mathcal{B}$ runs $Setup(\lambda)$ to obtain (SP, SK, PK) . $\mathcal{B}$ sends SP and all public keys in PK to $\mathcal{A}$.
- **Query:** $\mathcal{A}$ may adaptively issue metadata-generation queries. For each query, $\mathcal{A}$ submits a logical data block d to $\mathcal{B}$. Then $\mathcal{B}$ performs the following two algorithms internally.
 - $\mathcal{B}$ runs $PreGen(t)$ for some subtask t consistent with d .
 - $\mathcal{B}$ runs $MetaGen(M^*, d)$.

Then, $\mathcal{B}$ returns back to $\mathcal{A}$ the resulting auditing metadata M_d associated to the block d . It should be noticed that $\mathcal{B}$ simulates both $PreGen()$ and $MetaGen()$. internal details or formulas are omitted here, only the distribution is equivalent. $\mathcal{A}$ stores any data blocks and metadata as it wishes.

- **Challenge:** At any time, $\mathcal{B}$ may issue $Chal(|D| \rightarrow chal)$, on the set of data blocks previously queried by $\mathcal{A}$. The challenge $chal$ is sent to $\mathcal{A}$.
- **Forgery:** $\mathcal{A}$ responds with a proof $Prove(chal, D, M) \rightarrow P$. $\mathcal{A}$ wins if both conditions hold: (1) $Verify(chal, P) = T$; (2) $\mathcal{A}$ does not store all data blocks required by the challenge.
- **MODP-secure:** We say that MIDAS is MODP-secure if for every PPT adversary $\mathcal{A}$, $Pr[Game^{MODP}(\lambda) = 1] \leq negl(\lambda)$.

Metadata-outsourced Data Possession ensures that whenever an integrity proof is accepted, all data blocks covered by the challenge must be intact. To further capture that the entire outsourced file remains intact, we additionally introduce the notion of Detectability.

Definition 4 (Detectability): MIDAS is said to satisfy the detectability requirement when a TPA can reliably discover data integrity violations on the cloud server, even if the corruption affects only a limited portion of the stored data.

IV. DETAILED CONSTRUCTION OF MIDAS

To better illustrate the construction details of MIDAS, we first present the single-MU and single-EN version of MIDAS, which does not include the $TaskAssign(T, R) \rightarrow (A)$ algorithm. Then, we extend it to the multi-MU and multi-EN scenario by introducing the resource scheduling model to complete the $TaskAssign(T, R) \rightarrow (A)$ algorithm.

A. Basic Construction of MIDAS (Single-MU and Single-EN Setting)

In the Single-MU and Single-EN setting, MIDAS is presented in a simplified form, where a single MU relies on a single EN to

assist in generating its auditing metadata. In the next subsection, we extend this model to the multi-MU and multi-EN scenario, introducing the resource scheduling model and providing the detailed design of $TaskAssign(T, R) \rightarrow (A)$.

1) $Setup(\lambda) \rightarrow (SP, SK, PK)$: This algorithm is used for generating the system parameters and key pairs in MIDAS, and its detailed design is as follows:

- Given the security parameter λ , the MU initializes the system parameters as follows. Let $p = 2p' + 1$ and $q = 2q' + 1$ be two large safe primes, and define the RSA modulus as $N = pq$. A generator g of the quadratic residue subgroup μ_N is derived by setting $g = \rho^2$, where ρ is randomly selected from $\mathbb{Z}_N^*$ and satisfies $\gcd(\rho \pm 1, N) = 1$. The order of μ_N is defined as $|\mu_N|$.
- In addition, the MU specifies several common cryptographic primitives used in the system: a full-domain hash to μ_N $H(\cdot)$, a pseudorandom number generator $RG_\theta(\zeta, \eta)$ where θ is the random seed, ζ denotes the number of random outputs, and η defines their range, and a pair of digital signature algorithms, including the signature generation algorithm $S_{priv}(\cdot)$ with private signing key $priv$, and the signature verification algorithm $V_{pub}(\cdot)$ with corresponding public key pub .
- Then, all entities generate their own private and public keys.
 - The MU first selects a random element x from μ_N and computes $X = g^x \pmod{N}$. Then, the MU chooses a pair of parameters (e, w) satisfying the relation $ew \equiv 1 \pmod{p'q'}$, where e is kept locally. In addition, the MU generates a signature key pair (ssk_{mu}, spk_{mu}) . The private key set SK includes $\{x, e, ssk_{mu}\}$, while the corresponding public key set PK contains the remaining parameters.
 - The RA generates a signature key pair (ssk_{ra}, spk_{ra}) .
 - The EN determines its identity ID and generates a corresponding signature key pair (ssk_{en}, spk_{en}) .
 - The CS generates a signature key pair (ssk_{cs}, spk_{cs}) .
 - The TPA generates a signature key pair (ssk_{tpa}, spk_{tpa}) .

After the initialization phase, the set of system parameters SP is defined as $\{N, H(\cdot), RG_\theta(\zeta, \eta), S_{priv}(\cdot), V_{pub}(\cdot)\}$. The public key set PK is $\{X, w, ID, spk_{mu}, spk_{ra}, spk_{en}, spk_{cs}, spk_{tpa}\}$. All the remaining parameters belong to the private key set SK , which are securely kept by their corresponding entities.

For ease of exposition, the protocol description in this section assumes a single MU and a single EN. In practical deployments, the same procedures can be executed independently by multiple ENs in parallel on disjoint subsets of data blocks, as detailed in the multi-MU and multi-EN extension.

2) $PreGen(t) \rightarrow (M^*)$: Upon receiving the pre-metadata generation subtask assigned by the RA, the EN executes the following procedure:

- The EN extracts the specific content of the subtask $t = (FN, n, ind)$, where FN represents the filename of data, n denotes the number of pre-metadata items to be generated, and ind represents the starting index of the metadata. Since there is only one EN in the current setting, all pre-metadata

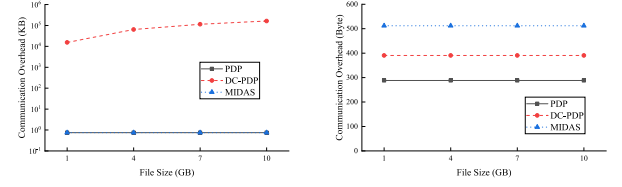


Fig. 7. Communication overhead comparison with different schemes (PDP, DC-PDP, MIDAS).

for the data blocks are generated by this single EN. Therefore, n equals the total number of data blocks, and ind is set to 0.

- The EN randomly selects two seeds θ_1 and θ_2 from $\mathbb{Z}_N^*$, and then uses the pseudorandom generator to produce two random number sets with n elements, denoted as $R^* = RG_{\theta_1}(n, N) = \{r_1^*, r_2^*, \dots, r_n^*\}$, $D^* = RG_{\theta_2}(n, N) = \{d_1^*, d_2^*, \dots, d_n^*\}$.
- The EN computes $B = \{b_1, b_2, \dots, b_n\}$, where each element is calculated as $b_i = H(FN || i) X^{r_i^*} g^{H(ID) \cdot d_i^*} \pmod{N}$.
- The EN generates a signature on θ_1 and θ_2 , denoted as $S_{ssk_{en}}(\theta_1 || \theta_2)$, and then sends $M^* = \{B, \theta_1, \theta_2\}$ to the MU.

It is worth noting that R^* and D^* are not transmitted directly. The MU can locally reconstruct R^* and D^* using the random seeds θ_1 and θ_2 , which significantly reduces the communication overhead at this stage, thus saving the cost of mobile data transmission. If the MU's computational resources are extremely limited, R^* and D^* can also be transmitted directly.

In addition, only θ_1 and θ_2 are signed rather than the entire set B . This design choice is made for two reasons: first, directly signing all elements of B would incur excessive computational cost; second, even if the signatures on B were valid, the MU would still need to perform an aggregated verification on M^* before use, since valid signatures on invalid B values would be meaningless. This consideration is particularly important because an EN may disappear after completing a single task and receiving its reward.

3) $MetaGen(M^*, D) \rightarrow (M)$: After receiving the pre-metadata M^* computed by the EN, the MU combines it with its own data D to be uploaded and generates the corresponding auditing metadata. Let $D = \{d_1, d_2, \dots, d_n\}$ denote the data blocks to be uploaded. The detailed procedure is as follows:

- The MU first verifies the signature $V_{spk_{en}}(\theta_1 || \theta_2)$. If the verification succeeds, it extracts the related information θ_1 and θ_2 .
- The MU reconstructs the random sets $R^* = RG_{\theta_1}(n, N) = \{r_1^*, r_2^*, \dots, r_n^*\}$ and $D^* = RG_{\theta_2}(n, N) = \{d_1^*, d_2^*, \dots, d_n^*\}$ using the pseudorandom generator.
- The MU performs an aggregated verification on M^* as follows: $\prod_{i=1}^n b_i \stackrel{?}{=} \prod_{i=1}^n (H(FN || i)) g^{\sum_{i=1}^n (x \cdot r_i^* + H(ID) \cdot d_i^*)} \pmod{N}$. If the verification fails, the protocol is terminated.

- The MU computes the auditing metadata for each data block, denoted as $B = \{b_1, b_2, \dots, b_n\}$, where each r_i is calculated as $r_i = (x \cdot r_i^* + H(ID) \cdot (d_i^* - d_i)) \cdot e \cdot x^{-1} \pmod{|\mu_N|}$, where x^{-1} denotes the modular inverse of x modulo $|\mu_N|$.
- The MU computes $g_{ID} = g^{H(ID) \cdot e} \pmod{N}$.
- The MU sends the auditing metadata $M = \{B, R\}$ together with the data D to the CS. Besides, The MU sends g_{ID} to TPA together with its signature.

It is worth noting that the MU is regarded as a trusted entity. Therefore, when transmitting the auditing metadata and data, the MU does not perform any signature operations. If the MU has sufficient computational resources, it may optionally sign the transmitted data. Alternatively, after receiving the metadata and data, the CS can perform a local aggregated verification, similar to how the MU verifies the EN's pre-metadata.

4) $Chal(|D|, g_{ID}) \rightarrow (chal)$: Next, the TPA periodically initiates data auditing on the CS. The process begins with the generation of an auditing challenge, and the detailed procedure is as follows:

- The TPA determines the number of challenged data blocks c based on the total number of data blocks $|D|$, where the ratio $\frac{c}{|D|}$ is typically a predefined constant.
- The TPA randomly selects an element s from $\mathbb{Z}_N^*$. Then, TPA computes $g_s = g_{ID}^s \pmod{N}$ and $X_s = X^s$.
- The TPA randomly selects two seeds α and β from $\mathbb{Z}_N^*$, which are used to determine the indices of the sampled data blocks and to generate the corresponding weighting coefficients for the proof.
- The TPA signs the challenge $chal = \{c, g_s, X_s, \alpha, \beta\}$ as $S_{ssk_{tpa}}(chal)$, and then sends it to the CS.

It is worth noting that determining the data block indices and random coefficients through the transmission of the random seeds α and β significantly reduces communication overhead and alleviates network bandwidth consumption. Meanwhile, generating the corresponding pseudorandom sequences incurs only negligible computational cost.

5) $Prove(chal, D, M) \rightarrow (P)$: Upon receiving the challenge $chal$ from the TPA, the CS generates the integrity proof by combining the currently stored data and auditing metadata. The detailed steps are as follows:

- The CS verifies the signature of $chal$, and then extracts the corresponding information c, g_s, X_s, α , and β .
- The CS computes index of the sampled data blocks and the corresponding coefficients, represented as $IND = RG_\alpha(c, n) = \{i_1, i_2, \dots, i_c\}$ and $COE = RG_\beta(c, N) = \{a_1, a_2, \dots, a_c\}$.
- The CS computes the integrity proof $P = \{\delta, \epsilon\}$, where $\delta = \prod_{j=1}^c b_{i_j}^{a_j} \pmod{N}$, $\epsilon = g_s^{\sum_{i=1}^c d_{i_j} \cdot a_j} \cdot X_s^{\sum_{j=1}^c r_{i_j} \cdot a_j} \pmod{N}$.
- The CS sends integrity proof $P = \{\delta, \epsilon\}$ with its signature to TPA.

6) $Verify(chal, P) \rightarrow (T/F)$: Once receiving the integrity proof P from the CS, the TPA verifies the integrity of the data stored on the CS based on the challenge $chal$. The detailed steps are as follows:

- After verifying the signature, the TPA extracts the specific information of the integrity proof, namely $\{\delta, \epsilon\}$.
- The TPA computes the sampled data block indices and the corresponding random coefficients using the random seeds, where $IND = RG_\alpha(c, n) = \{i_1, i_2, \dots, i_c\}$ and $COE = RG_\beta(c, N) = \{a_1, a_2, \dots, a_c\}$.
- The TPA verifies the proof by checking $\delta^s \pmod{N} \stackrel{?}{=} \prod_{j=1}^c (H(FN || i_j)^{s \cdot a_j}) \epsilon^w \pmod{N}$. If the equation holds, it indicates that the cloud data remains intact and the TPA outputs T ; otherwise, F is returned.
- The TPA periodically reports the auditing verification results to the MU.

B. Extension to Multi-MU and Multi-EN: Resource Scheduling Model

In the multi-MU and multi-EN scenario of MIDAS, the protocol described for the single-MU and single-EN setting naturally extends by executing the same pre-metadata generation procedure in parallel on multiple ENs. Specifically, the metadata generation task of each MU is split at the block level, and different subsets of blocks are assigned to multiple ENs for parallel execution. The MU subsequently aggregates and verifies the pre-metadata following the same verification and metadata generation steps as in the single-EN case.

The goal of the RA is to distribute these block-level tasks across heterogeneous ENs such that the maximum completion time among all ENs is minimized. To this end, we first normalize the computation cost under different security levels, and then formulate a load-balanced scheduling model.

1) *Task Modeling and Workload Normalization*: In this work, we focus on the computation overhead of pre-metadata generation, which dominates the task execution time. Other overheads, such as communication and coordination, are relatively small or approximately uniform across edge nodes and thus are omitted from the scheduling model.

Let $\mathcal{U} = \{MU_1, MU_2, \dots, MU_M\}$ denote the set of mobile users. Each MU_i has one metadata-generation task associated with a file of size F_i . Let β_i be the data block size chosen by MU_i , and thus the number of blocks is $n_i = \lceil \frac{F_i}{\beta_i} \rceil$.

Each task i is assigned a security level σ_i (e.g., 1024-bit, 2048-bit RSA). Let $c(\sigma_i)$ denote the average computation cost of generating one auditing tag b under security level σ_i . Clearly, stronger security levels incur larger $c(\sigma_i)$.

To unify heterogeneous-security tasks in a single framework, we choose a baseline security level σ_0 (e.g., 1024-bit RSA) with per-tag cost $c_0 = c(\sigma_0)$, and define the normalization factor $\alpha(\sigma_i) = \frac{c(\sigma_i)}{c_0}$. Intuitively, $\alpha(\sigma_i)$ represents how many baseline-security tags are equivalent (in terms of computation cost) to one tag at security level σ_i .

The normalized computational workload of task i is then defined as

$$W_i = n_i \cdot \alpha(\sigma_i), \quad (1)$$

which can be interpreted as the equivalent number of baseline-security tags required to generate all auditing tags for MU_i .

The total normalized workload in the system is $W_{\text{tot}} = \sum_{i=1}^M W_i$.

2) *Edge Node Capacity Modeling*: Let $\mathcal{E} = \{EN_1, EN_2, \dots, EN_N\}$ denote the set of edge nodes. Each EN_j is equipped with a certain computational capability, measured as the number of baseline-security tags it can generate per unit time. We denote this capacity by $f_j > 0$. The total normalized computation capacity of all ENs is $F_{\text{tot}} = \sum_{j=1}^N f_j$.

Given that metadata generation is performed at the block level, the task of MU_i can be split among multiple ENs. Let $n_{i,j}$ denote the number of blocks of task i assigned to EN_j . We then have the block-level conservation constraint

$$\sum_{j=1}^N n_{i,j} = n_i, \quad n_{i,j} \in \mathbb{Z}_{\geq 0}, \quad \forall i. \quad (2)$$

The normalized workload assigned to EN_j is

$$L_j = \sum_{i=1}^M n_{i,j} \cdot \alpha(\sigma_i), \quad (3)$$

which indicates how many baseline-equivalent tags EN_j needs to generate.

Since f_j is the rate (baseline tags per unit time), the completion time of EN_j is approximated by

$$C_j = \frac{L_j}{f_j}. \quad (4)$$

3) *Load-Balanced Scheduling Objective*: Under this model, the scheduling problem aims to minimize the maximum completion time among all ENs, i.e.,

$$\min \max_{1 \leq j \leq N} C_j = \min \max_{1 \leq j \leq N} \frac{L_j}{f_j}, \quad (5)$$

subject to

$$\sum_{j=1}^N n_{i,j} = n_i, \quad \forall i = 1, 2, \dots, M, \quad (6)$$

$$n_{i,j} \in \mathbb{Z}_{\geq 0}, \quad \forall i, j. \quad (7)$$

In the ideal fractional case (ignoring integrality), the optimal solution is achieved when all ENs finish at the same time, i.e., $C_1 = C_2 = \dots = C_N = \frac{W_{\text{tot}}}{F_{\text{tot}}}$, which corresponds to assigning workloads proportionally to capacities, that is, $L_j^* = W_{\text{tot}} \cdot \frac{f_j}{F_{\text{tot}}}$.

In practice, since $n_{i,j}$ must be integers and constrained by n_i , we adopt a simple proportional block allocation strategy in our scheduling algorithm: for each task i , its n_i blocks are distributed across ENs such that $n_{i,j} \approx n_i \cdot \frac{f_j}{F_{\text{tot}}}$, while ensuring the constraints in (6) and (7). This proportional assignment naturally approximates the ideal balanced condition and thus minimizes the maximum completion time among all ENs under the normalized workload model.

Algorithm 1: Proportional Load-Balanced Scheduler.

Input: File sizes $\{F_i\}_{i=1}^M$, block sizes $\{\beta_i\}_{i=1}^M$, security levels $\{\sigma_i\}_{i=1}^M$, normalization factors $\{\alpha(\sigma_i)\}_{i=1}^M$, EN capacities $\{f_j\}_{j=1}^N$.

Output: Block assignment $\{n_{i,j}\}$ for all tasks i and edge nodes j .

for $i \leftarrow 1$ **to** M **do**

$n_i \leftarrow \left\lceil \frac{F_i}{\beta_i} \right\rceil$;
 $W_i \leftarrow n_i \cdot \alpha(\sigma_i)$;

$W_{\text{tot}} \leftarrow \sum_{i=1}^M W_i$;

$F_{\text{tot}} \leftarrow \sum_{j=1}^N f_j$;

for $i \leftarrow 1$ **to** M **do**

for $j \leftarrow 1$ **to** N **do**

$n_{i,j} \leftarrow \left\lfloor n_i \cdot \frac{f_j}{F_{\text{tot}}} \right\rfloor$;

$r_i \leftarrow n_i - \sum_{j=1}^N n_{i,j}$;

while $r_i > 0$ **do**

Select $j^* \in \{1, \dots, N\}$ with the largest f_{j^*} ;

$n_{i,j^*} \leftarrow n_{i,j^*} + 1$;

$r_i \leftarrow r_i - 1$;

return $\{n_{i,j}\}$;

C. Proportional Load-Balanced Scheduling Algorithm

To operationalize the scheduling model described above, MIDAS adopts a lightweight proportional load-balanced scheduling (PLS) algorithm executed by the RA, which is shown in Algorithm 1.

The key idea is to leverage the normalized workload representation and allocate block-level metadata generation tasks to ENs in proportion to their baseline-normalized computational capacities. Since metadata generation for each MU can be split across multiple ENs, the proportional allocation naturally approximates the optimal solution of the $\min \max_j C_j$ objective, ensuring that all ENs complete their assigned workloads within nearly the same time. The algorithm operates with only $O(MN)$ time complexity and introduces negligible computational overhead, making it suitable for deployment in practical mobile-edge environments.

V. SECURITY ANALYSIS

This section provides a security analysis based on the proposed security model.

A. Correctness

Theorem 1: If all participating entities follow the MIDAS protocol correctly, the integrity proof produced by the CS will consistently be accepted during the TPA's verification process.

Proof: The proof information of the related blocks is denoted as $P = \{\delta, \epsilon\}$. The TPA needs to verify whether $\delta^s \pmod{N} \stackrel{?}{=} \prod_{j=1}^c (H(FN \| i_j)^{s \cdot a_j}) \epsilon^w \pmod{N}$ holds. Let $\prod_{j=1}^c H(FN \| i_j)^{s \cdot a_j} = \tau \pmod{N}$. The verification process

is shown as follows:

$$\begin{aligned}
& \tau \epsilon^w \\
&= \tau \cdot \left(g_s^{\sum_{i=1}^c d_{ij} \cdot a_j} \cdot X_s^{\sum_{j=1}^c r_{ij} \cdot a_j} \right)^w \\
&= \tau \cdot \left(g^{H(ID) \sum_{i=1}^c a_j \cdot d_{ij}} \right)^{e \cdot w \cdot s} \cdot \left(g^{\sum_{j=1}^c a_j \cdot r_{ij}} \right)^{x \cdot w \cdot s} \\
&= \tau \cdot \left(g^{H(ID) \sum_{i=1}^c a_j \cdot d_{ij}} \right)^s \\
&\quad \times \left(g^{\sum_{j=1}^c a_j \cdot (x \cdot r_{ij}^* + H(ID) \cdot (d_{ij}^* - d_{ij})) \cdot e \cdot x^{-1}} \right)^{x \cdot w \cdot s} \\
&= \tau \cdot \left(g^{\sum_{j=1}^c (H(ID) \cdot d_{ij} + x \cdot r_{ij}^* + H(ID) \cdot (d_{ij}^* - d_{ij}))} \right)^s \\
&= \tau \cdot \left(g^{\sum_{j=1}^c a_j \cdot (x \cdot r_{ij}^* + H(ID) \cdot d_{ij}^*)} \right)^s \\
&= \prod_{j=1}^c H(FN \| i_j)^{s \cdot a_j} \cdot \left(g^{\sum_{j=1}^c (x \cdot r_{ij}^* + H(ID) \cdot d_{ij}^*)} \right)^s \\
&= \left(\prod_{j=1}^c (X^{r_{ij}^*} \cdot g^{H(ID) \cdot d_{ij}^*})^{a_j} \right)^s \\
&= \delta^s
\end{aligned}$$

□

It should be noted that all derivations are conducted under $(\text{mod } N)$, and the modulus is omitted throughout for simplicity. Moreover, the key transformation occurs in Step 3, where $r_{ij} = (x \cdot r_{ij}^* + H(ID) \cdot (d_{ij}^* - d_{ij})) \cdot e \cdot x^{-1}$ is substituted.

B. Metadata-Outsourced Data Possession

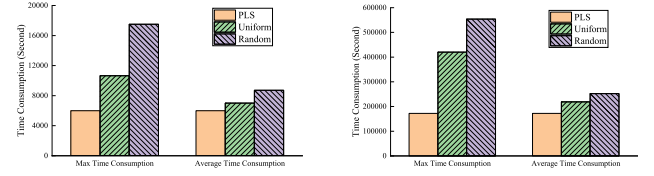
Theorem 2: Assume that the hash function $H(\cdot)$ is regarded as a random oracle. If the DL problem in QR_N is hard (equivalently, if factoring N is hard), then MIDAS is provably secure under the $\text{Game}^{\text{MODP}}$ security model, with a reduction loss of $L = 1$.

Proof: The reduction $\mathcal{R}$ receives a DL instance $(g, X = g^x) \in QR_N$ and aims to recover x . $\mathcal{R}$ simulates the MU and the TPA, while the adversary $\mathcal{A}$ controls the EN and the CS.

Setup embedding: $\mathcal{R}$ sets the public generator to g and the MU's public key to $X = g^x$. It samples $e \in \mathbb{Z}_{p'q'}^*$ and sets $w \equiv e^{-1} \text{ mod } p'q'$. It further defines $g_{ID} = g^{H(ID)e}$. The hash function $H(\cdot)$ is modeled as a random oracle mapping into QR_N .

Query simulation: For each metadata-generation query of $\mathcal{A}$ on block index i , $\mathcal{R}$ samples $r_i \leftarrow \mathbb{Z}_{p'q'}$ uniformly at random and sends r_i to $\mathcal{A}$. Internally, $\mathcal{R}$ samples $r_i^*, d_{ij}^* \leftarrow \mathbb{Z}_{p'q'}$ uniformly and computes $b_i = H(FN \| i) \cdot X^{r_i^*} \cdot g^{H(ID)d_{ij}^*} \text{ (mod } N)$. It then returns b_i to $\mathcal{A}$. Although the simulator samples $\{r_i\}$ uniformly at random, the joint distribution of (B, R) is identical to that in the real protocol from $\mathcal{A}$'s point of view, since the defining relation of r_i in MetaGen depends only on the MU's secret values (x, e) and the hidden randomness (r_i^*, d_{ij}^*) . Hence, in the following analysis we may reason about the corresponding ideal execution where the MetaGen relation holds.

Challenge and forking: $\mathcal{R}$ chooses $s \in \mathbb{Z}_{p'q'}^*$ and seeds (α, β) , computes $IND = RG_\alpha(c, n)$ and $COE = RG_\beta(c, N)$, and sets $g_s = g_{ID}^s$ and $X_s = X^s$. Let $chal = \{c, g_s, X_s, \alpha, \beta\}$.



(a) Maximum and Average completion time in small-scale Scenario

(b) Maximum and Average completion time in large-scale Scenario

Fig. 8. Comparison of scheduling results under different strategies (PLS, Uniform, Random).

The adversary $\mathcal{A}$ outputs a valid proof $P = \{\delta, \epsilon\}$ satisfying $\delta^s \equiv (\prod_{j=1}^c H(FN \| i_j)^{s a_j}) \epsilon^w \text{ (mod } N)$. The reduction $\mathcal{R}$ then rewinds $\mathcal{A}$ to the point before s is chosen, keeps the random oracle state and (α, β) unchanged, and replaces s with an independently sampled $s' \neq s$. With non-negligible probability, $\mathcal{A}$ outputs a second valid proof $P' = \{\delta', \epsilon'\}$.

Algebraic cancellation: Dividing the two verification equations yields $(\epsilon'/\epsilon)^w = \delta^{\Delta s} (\prod_{j=1}^c H(FN \| i_j)^{\Delta s a_j})^{-1} \text{ (mod } N)$, where $\Delta s = s' - s$, which is invertible modulo $p'q'$ with overwhelming probability. Let $S = \sum_{j=1}^c a_j d_{ij}$, $S^* = \sum_{j=1}^c a_j d_{ij}^*$, $R = \sum_{j=1}^c a_j r_{ij}$, and $R^* = \sum_{j=1}^c a_j r_{ij}^*$. Substituting the definitions of δ and ϵ , and using $g_{ID} = g^{H(ID)e}$ and $w = e^{-1}$, we obtain $g^{H(ID)S} \cdot X^{wR} = X^{R^*} \cdot g^{H(ID)S^*} \text{ (mod } N)$. Rearranging terms gives $X^{wR - R^*} = g^{H(ID)(S^* - S)} \text{ (mod } N)$. Define $K := wR - R^*$ and $L := H(ID)(S^* - S)$. Thus we derive the key extraction equation $X^K = g^L \text{ (mod } N)$.

Recovering x : In the ideal execution of the protocol, the MU computes $r_i \equiv (x r_i^* + H(ID)(d_{ij}^* - d_{ij})) e x^{-1} \text{ (mod } p'q')$. Multiplying both sides by $w = e^{-1}$ and summing over the challenge indices yields $wR \equiv R^* + H(ID)x^{-1}(S^* - S) = R^* + x^{-1}L \text{ (mod } p'q')$. Hence $K = wR - R^* \equiv x^{-1}L \text{ (mod } p'q')$. Since $x \in (\mathbb{Z}_{p'q'})^*$, K is invertible if and only if L is invertible. When this holds, combining $X = g^x$ with $X^K = g^L$ yields $x \equiv L \cdot K^{-1} \text{ (mod } p'q')$.

Success probability and complexity: The forking step incurs only a constant-factor loss. The value Δs is invertible modulo $p'q'$ with probability at least $1 - 1/p' - 1/q'$. Moreover, $L = H(ID)(S^* - S)$ is nonzero and invertible with non-negligible probability whenever $\mathcal{A}$ successfully forges a proof without storing all challenged blocks. Therefore, $\mathcal{R}$ recovers x with non-negligible probability and runs in probabilistic polynomial time.

Conclusion: Thus $\mathcal{R}$ computes $\log_g X = x$ in QR_N . Consequently, any adversary that wins $\text{Game}^{\text{MODP}}$ yields a solver for the DL problem in QR_N , which is polynomial-time equivalent to factoring N . □

C. Detectability

Theorem 3: Even when a minor portion of data on the CS is compromised, the TPA can detect such anomalies with non-negligible probability.

Proof: Assume that the CS removes k data blocks from a file composed of n blocks, and let c represent the number

of blocks involved in a challenge. The probability that such data loss can be detected is denoted by $P_{detected}$, which is given by $P_{detected} = 1 - P_{undetected} = 1 - \frac{n-k}{n} \times \frac{n-k-1}{n-1} \times \frac{n-k-2}{n-2} \times \dots \times \frac{n-k-c+1}{n-c+1}$. Since $\frac{n-i-k}{n-i} > \frac{n-i-1-k}{n-i-1}$ holds for all valid i , it follows that $1 - (\frac{n-k}{n})^c < P_{detected} < 1 - (\frac{n-k-c+1}{n-c+1})^c$. In practical scenarios, the resulting detection probability is sufficiently large to be considered non-negligible. From an intuitive perspective, this analysis shows that when the proportion of corrupted data reaches 1%, auditing roughly 4.6% of the data blocks is adequate to ensure a detection probability greater than 99%. $\square$

VI. PERFORMANCE EVALUATION

This section presents a comprehensive performance evaluation of MIDAS from two perspectives: Single-MU and Single-EN, and Multi-MU and Multi-EN. The former scenario is designed to demonstrate the various types of overhead incurred by MIDAS when processing real-world binary data across different execution stages. The latter focuses on validating the proposed resource scheduling algorithm under complex auditing metadata outsourcing requirements.

A. Experiment Setup for Single-MU and Single-EN MIDAS

The experimental environment consists of four roles deployed on heterogeneous platforms. The MU is an iPhone 15 Pro equipped with 8 GB RAM and an A17 Pro processor, with IP address 192.168.3.1. The CS is hosted on Huawei Cloud and configured with 16 GB RAM and 8 virtual CPUs at 3.0 GHz, using the public IP address 1.94.176.190. The TPA and RA run on a Huawei MateBook with 16 GB RAM and an Intel i7-1165G7 processor, assigned IP address 192.168.3.2. The EN is deployed on an ROG SCAR 17 system equipped with 64 GB RAM and an Intel i9-13980HX processor, with IP address 192.168.3.3.

All algorithms were implemented in Python. PyCharm was employed as the development environment on desktop computers, while Pythonista was used on the iPhone platform. To emulate an edge computing scenario, a local area network (LAN) was constructed using a local router. Communication between the storage server and the edge network was realized through a public IP address allocated by Huawei Cloud, thereby approximating a realistic cloud-edge interaction environment.

To eliminate differences in data formats such as video, images, and text, all data are in the form of byte stream files, specifically in ".bin" file format. File sizes are 1 GB, 4 GB, 7 GB, and 10 GB, with data block sizes of 4 KB, 8 KB, 16 KB and 32 KB.

Two server-oriented data auditing schemes were selected as baselines for comparison. One is the classical scheme, denoted as PDP [13], and the other is the latest scheme, denoted as DC-PDP [33]. According to *Theorem 3*, the sample ratio is 4.6%. Both PDP and MIDAS adopt the RSA cryptosystem, where the security parameter N is set to 2048 bits. DC-PDP is implemented over a cyclic group, and following the settings in its original paper, q' is set to 160 bits, q is set to 512 bits, and p is set to 1024 bits. SHA-256 is adopted as the hash function, while

data confidentiality is ensured using the Advanced Encryption Standard (AES) with a 256-bit key.

In our implementation, bilinear pairing-related group operations were implemented using the pypbc library. RSA- and AES-based cryptographic primitives were instantiated using Py-Cryptodome Cryptographic. Hash functions were implemented using Python's built-in hashlib (e.g., SHA-256). Data transmission among different entities was realized using Python's standard TCP/IP socket interfaces, which provide reliable communication.

B. Experiment Results for Single-MU and Single-EN MIDAS

This subsection presents the comparison results of computational and communication overhead at different stages.

1) *Performance Analysis of EN for PreGen*: The PreGen phase is unique to MIDAS, as PDP and DC-PDP do not support outsourced generation of auditing metadata. Therefore, the computational and communication overhead in this phase is reported only for MIDAS. The experimental results are shown in Fig. 2.

As shown in Fig. 2, both the computational overhead and the communication overhead exhibit the same trend. This is because the PreGen phase performs identical operations on each data block, and all generated pre-metadata are computed modulo the security parameter N , which implies that each pre-metadata item has a fixed size. Consequently, the computational and communication costs of PreGen scale with the total number of data blocks. For a fixed block size, a larger file results in more blocks, which explains the linear increase in overall overhead with respect to file size observed in both Fig. 2(a) and (b).

When the file size is fixed, the results increase significantly as the block size decreases (32 KB, 16 KB, 8 KB, and 4 KB). In particular, the computational and communication overhead with a 32 KB block size is approximately twice that with a 16 KB block size. This is because the number of data blocks is calculated as $\frac{\text{file size}}{\text{block size}}$, indicating an inverse relationship between the number of data blocks and the block size. The computational overhead at the EN is mainly determined by $b_i = H(FN||i)X_i^{r_i^*}g^{H(ID) \cdot d_i^*} \pmod{N}$, where r_i^* and d_i^* are random values generated by a pseudorandom function, and their lengths are directly related to the security parameter. The per-block runtime of PreGen is dominated by modular exponentiations in computing $X_i^{r_i^*}$ and $g^{H(ID) \cdot d_i^*}$, while pseudorandom function and hashing contribute only a minor portion. Therefore, the total time scales almost linearly with the number of blocks. Consequently, although the block size varies, the time required to compute each pre-metadata item remains unchanged. The communication overhead follows the same pattern, since the storage size required for each pre-metadata item is fixed.

By adopting secure outsourcing of auditing metadata, MIDAS significantly reduces computational overhead at the EN. The resulting communication overhead is acceptable in practice: for a 10 GB file, it amounts to only several hundred megabytes and can be further reduced by increasing the block size.

2) *Performance Analysis of MU for MetaGen*: The comparative results of the MetaGen phase are illustrated in Figs. 3 and 4.

It should be noted that, due to the high computational overhead of PDP and the excessive communication overhead of DC-PDP, logarithmic scales are adopted to provide a clearer comparison between MIDAS and the two baseline schemes.

As shown in Fig. 3, the experimental results of the MetaGen phase indicate that MIDAS consistently achieves the lowest time overhead, regardless of variations in data block size or file size. For a 10 GB file with a block size of 4 KB, the computation time at the MU side for MIDAS is slightly over 13 minutes, while it is reduced to just over 7 minutes when the block size increases to 32 KB. These results demonstrate that MIDAS effectively addresses the challenge of auditing metadata generation on resource-constrained devices. A detailed analysis of each scheme is provided as follows.

For PDP, homomorphic verification tags must be generated directly on the mobile device, which is an extremely time-consuming operation and leads to prohibitively high computational overhead. In our experiments, generating tags for a 10 GB file on an iPhone 15 Pro requires more than two days. Given that a mobile phone can sustain high-load computation for only about six hours on a single battery charge, PDP is clearly impractical for direct deployment on mobile devices. Specifically, when the data block size is fixed, the computational overhead of PDP increases with the file size. Although this growth appears as a curve in the figures due to the use of logarithmic scales, the actual trend is linear. This is because the time required to generate metadata for each data block is constant, making the overall cost directly proportional to the number of data blocks. Moreover, when the file size is fixed, a horizontal comparison across the four subfigures shows that larger block sizes result in lower computational overhead. This is because larger data blocks lead to fewer total blocks, while the time to generate metadata for different block sizes remains similar. PDP can reduce the exponent to the same 2048-bit level via the $\text{mod } \phi(N)$ operation, and the exponentiation dominates the overall metadata generation cost.

For DC-PDP and MIDAS, only simple arithmetic operations are performed on mobile devices, resulting in significantly lower computational overhead compared to PDP. In addition, the metadata computation cost of DC-PDP is independent of the data block size, since the block size in DC-PDP cannot exceed 160 bits, i.e., the size of q . This limitation causes DC-PDP to process tens of times more data blocks than MIDAS and PDP, specifically by a factor of $\frac{\text{blocksize}}{q}$.

For MIDAS, metadata generation consists solely of lightweight arithmetic operations and is therefore insensitive to the file block size. Concretely, the MU-side cost is mainly incurred by the aggregated verification of M^* and the per-block computation of r_i , both of which involve modular multiplications/additions and scale with the number of blocks, whereas signature verification and pseudorandom function reconstruction are one-time overheads. Accordingly, the computational cost of MIDAS is governed by the total number of data blocks. With a fixed block size, larger files incur higher overhead, whereas for a fixed file size, increasing the block size leads to reduced computational overhead. It is worth noting that the maximum block size in the experiments is limited to 32 KB. In practice,

MIDAS supports even larger block sizes, which further amplifies its performance advantages when handling large-block files.

Fig. 4 presents a comparison of the communication and storage overhead of the three schemes in the MetaGen phase. Because the auditing metadata generated at this stage must be transmitted and persistently stored, these two types of overhead are inherently identical and are thus evaluated jointly.

Among the three schemes, DC-PDP clearly introduces the largest communication/storage overhead, amounting to roughly two to three times the size of the original file. This inefficiency arises from its auditing design, which transforms each q -bit data block into an encoded block of size q' . Following the original parameter settings in our experiments ($q = 160$ bits and $q' = 512$ bits), the resulting expansion factor reaches approximately 2.2, i.e., $\frac{q'-q}{q}$. In a wired edge network equipped with gigabit routing and network interfaces, such overhead remains manageable. However, this assumption no longer holds for mobile devices operating over cellular networks, where bandwidth is severely constrained. For instance, under a typical 4G connection, even with strong signal quality, uplink throughput is usually limited to about 10 MB/s (80 Mb/s). At this rate, uploading the additional 22 GB required for a 10 GB file would take roughly 37 minutes. Such prolonged transmissions not only impose excessive energy consumption but also make the process impractical for battery-powered devices. In addition, cellular data usage incurs monetary costs, and the $2.2\times$ increase in server-side storage further translates into higher cloud service fees. Consequently, despite its moderate computational overhead, DC-PDP suffers from prohibitive communication and storage costs.

By contrast, PDP and MIDAS incur communication overhead solely from the auditing metadata generated at the mobile device. Each data block contributes one or more metadata items, yielding an overall overhead of $\frac{\text{filesize}}{\text{blocksize}} \times \lambda$, where λ denotes the security parameter and is set to 2048 bits in our experiments. For a 10 GB file with a 4 KB block size, PDP and MIDAS generate approximately 640MB and 1280MB of communication overhead, respectively—orders of magnitude lower than the 22 GB incurred by DC-PDP. Moreover, increasing the block size leads to a rapid reduction in overhead for both schemes. When the block size reaches 32 KB, the overhead drops to only 32MB for PDP and 64MB for MIDAS. This demonstrates the clear communication and storage efficiency of PDP and MIDAS during metadata generation. Although MIDAS incurs roughly twice the overhead of PDP due to producing two metadata parameters per block, this increase remains well within practical limits.

In summary, when both computational requirements and communication/storage costs are taken into account, MIDAS is the only scheme that can be realistically deployed on mobile devices, enabling secure and efficient auditing metadata outsourcing under resource-constrained mobile settings.

3) *Performance Analysis of Audit*: Figs. 5, 6, and 7 present the performance evaluation of the Audit phase, including Chal, Prove and Verify. Specifically, Fig. 5 depicts the computational cost incurred by the CS when generating audit proofs, Fig. 6 reports the time overhead of the TPA for challenge generation and proof verification, and Fig. 7 summarizes the communication

overhead between the CS and the TPA throughout the auditing procedure. It is worth noting that the MU is no longer involved in the Audit phase. As a result, when the objective is limited to secure metadata outsourcing on resource-constrained mobile devices, the results of this phase are not critical. Nonetheless, to ensure a thorough assessment of MIDAS, we still conduct detailed comparative experiments for the Audit phase.

As illustrated in Fig. 5, MIDAS incurs the smallest computational cost among the three schemes. The dominant overhead in the Audit phase mainly arises from two factors: accessing the data blocks specified by the audit challenge and performing exponentiation operations during proof construction.

For DC-PDP, the block size is fixed at 160 bits, resulting in a number of data blocks that is tens of times larger than that of PDP and MIDAS. Consequently, locating and retrieving the challenged blocks from such a large dataset is highly expensive, which significantly increases its computational overhead.

In contrast, the computation cost of PDP and MIDAS is primarily determined by the number of exponentiation operations involved in proof generation. Due to the use of random coefficients in the challenge, PDP must perform exponentiations on both metadata and multiple hash values, whereas MIDAS only requires exponentiation on the metadata. This design difference explains why MIDAS consistently outperforms PDP in terms of computational efficiency.

Moreover, PDP and MIDAS demonstrate similar scalability behavior. Since the sampling ratio is fixed at 4.6% of the total number of data blocks, the proof-generation overhead in both schemes increases with the total block count. As a result, for a fixed block size, larger files lead to higher computational overhead, while for a fixed file size, increasing the block size reduces the overhead. By comparison, the computational cost of DC-PDP remains largely unchanged with respect to block size, as it always operates on a fixed block length of 160 bits.

As illustrated in Fig. 6, DC-PDP generally exhibits lower computational overhead at the TPA than PDP and MIDAS. This is mainly because verification in PDP and MIDAS involves exponentiation operations on hash values for the challenged indices, whereas DC-PDP relies only on lightweight aggregation and basic hash computations. Compared with PDP, MIDAS further reduces the verification cost by avoiding the computation of inverse hash exponentiations, which explains its lower overhead. Notably, when the block size exceeds 16 KB, MIDAS outperforms DC-PDP in terms of computational efficiency, demonstrating the benefit of adopting the RSA-based design.

Moreover, as the number of hash exponentiations increases with the total number of data blocks, the overall performance trend follows the same pattern observed in Fig. 5 and is therefore omitted for brevity.

As depicted in Fig. 7(a), the communication overhead of MIDAS and PDP at the TPA side is approximately 1 KB, which is negligible in practice. In contrast, DC-PDP incurs substantially higher communication overhead. This is because DC-PDP must explicitly transmit the index and coefficient sets, whose construction requires carefully selected coefficients. Combined with the large number of data blocks involved, this design leads to significant communication cost. By comparison, PDP generates

random indices and coefficients using pseudorandom functions and only needs to transmit two random seeds along with a random parameter g_s , resulting in a communication cost of $3 \times \lambda$. Moreover, while the communication overhead of PDP and MIDAS remains independent of the file size, that of DC-PDP increases as the file size grows due to the expanding index and coefficient sets and the increasing bit length required to represent indices.

Fig. 7(b) shows that MIDAS introduces the largest communication overhead in proof transmission, as each proof consists of two parameters of size $\lambda = 2048$ bits. In contrast, PDP returns only one proof value (2048 bits) and one hash value (256 bits). For DC-PDP, all sampled encoded blocks must be aggregated and transmitted, causing its communication overhead to grow slightly with the file size.

Overall, during the Audit phase, MIDAS achieves lower computational overhead at the storage server and lower communication overhead at the TPA compared with the other schemes. When the block size exceeds 16 KB, MIDAS also incurs the lowest computational cost at the TPA. Although MIDAS does not minimize the TPA-side communication overhead, the cost remains below 1 KB and is therefore acceptable. Taken together, MIDAS consistently outperforms the other two schemes in the Audit phase.

C. Experiment Setup for Scheduling Algorithm (Multi-MU and Multi-EN MIDAS)

To evaluate the proposed scheduling algorithm, we conduct simulations under two representative settings: a small-scale environment and a large-scale environment. All metadata-generation tasks are modeled at the block level, and computation costs are normalized with respect to the 1024-bit security level.

1) *Small-Scale Scenario*: We consider $M = 10$ MUs and $N = 3$ ENs. Each MU_i is assigned a file whose size F_i is randomly sampled from $U(50\text{MB}, 500\text{MB})$. The block size is fixed to 4 KB, and the number of blocks is $n_i = \lceil F_i / 4 \text{ KB} \rceil$. The security level $\sigma_i \in \{1024, 2048, 3072\}$ is selected uniformly at random. The corresponding normalization factors are $\alpha(1024) = 1$, $\alpha(2048) = 4$, $\alpha(3072) = 9$, and the normalized workload is $W_i = n_i \alpha(\sigma_i)$. Three heterogeneous ENs are configured with baseline capacities $f_1 = 256$, $f_2 = 160$, and $f_3 = 96$ (in normalized 1024-bit operations per second), representing strong, medium, and weak nodes respectively.

2) *Large-Scale Scenario*: We consider $M = 50$ MUs and $N = 5$ ENs. File sizes are sampled from $U(100\text{MB}, 10 \text{ GB})$, with the same 4 KB block size and the same definition of n_i and W_i . Security levels follow a non-uniform distribution: $\Pr(\sigma_i = 1024) = 0.5$, $\Pr(\sigma_i = 2048) = 0.4$, and $\Pr(\sigma_i = 3072) = 0.1$. Five ENs with heterogeneous capacities are used: $f_1 = 320$, $f_2 = 256$, $f_3 = 192$, $f_4 = 128$, and $f_5 = 80$.

3) *Compared Schemes and Metrics*: We compare three scheduling schemes: the proposed proportional scheduler (PLS), Uniform splitting, and Random assignment. For each scheme, the normalized load assigned to EN j is $L_j = \sum_i n_{i,j} \alpha(\sigma_i)$, and its completion time is $C_j = L_j / f_j$. Two metrics were adopted:

- *Maximum completion time*: $C_{\max} = \max_j C_j$, which reflects the overall task finishing time.

- *Average completion time:* $\bar{C} = \frac{1}{N} \sum_{j=1}^N C_j$, which measures the mean latency across all ENs.

These two metrics together characterize both the worst-case delay and the overall scheduling efficiency.

D. Experiment Results for Scheduling Algorithm (Multi-MU and Multi-EN MIDAS)

The experimental comparison of the resource scheduling algorithm for Multi-MU and Multi-EN MIDAS is shown in Fig. 8.

As shown in Fig. 8, the proposed PLS consistently outperforms the other two baseline algorithms in terms of both maximum and average completion time, across both small-scale and large-scale application scenarios. Moreover, the maximum and average completion times achieved by PLS are identical, indicating that PLS attains the optimal scheduling solution. This is mainly because PLS computes separate normalization factors for different security levels, allowing each MU's computation workload to be decomposed into fundamental subtasks, specifically the baseline time required to generate pre-metadata for a 4 KB data block at the 1024-bit security level. Based on this baseline, pre-metadata generation tasks are then proportionally assigned to ENs according to their respective computational capacities. In this way, PLS enables resource scheduling that is aligned with the computational requirements of metadata generation.

In contrast, the Uniform scheduling strategy distributes pre-metadata computation tasks evenly among ENs. However, it fails to account for heterogeneity in EN computational capabilities as well as variations in the computational costs of different pre-metadata tasks. Consequently, this strategy may lead to inefficient resource utilization and performance degradation under heterogeneous workloads. The Random scheduling strategy further neglects both task complexity and EN computing capacity, relying solely on random task assignment, which results in severe load imbalance and renders it the least effective approach among the three scheduling strategies.

VII. CONCLUSION

Due to the limited computational capability of mobile devices, efficient auditing metadata generation remains a major barrier to enabling periodic cloud data auditing. To address this issue, this paper proposes MIDAS, a mobile-friendly data auditing scheme that supports secure outsourcing of auditing metadata while achieving balanced resource utilization. MIDAS offloads computation-intensive operations to edge nodes and retains only lightweight processing on mobile devices, thereby significantly reducing client-side overhead. In addition, identity information of the edge nodes is embedded into the auditing metadata to support traceability and accountability, and the security of the outsourcing mechanism is formally established via a reduction to the discrete logarithm problem in the RSA quadratic residue subgroup. Building on this design, the PLS algorithm is further proposed to enable efficient resource scheduling in Multi-MU and Multi-EN scenarios. Experimental results in a real-world environment show that MIDAS introduces the lowest computational overhead on mobile devices, with communication cost

remaining acceptable, while PLS achieves optimal completion time compared with baseline methods. Overall, MIDAS enables continuous and trustworthy cloud data auditing for resource-constrained mobile users without compromising device performance or user experience.

REFERENCES

- [1] R. Geuens, "What percentage of internet traffic is mobile?," SOAX Research, Jul. 2025. Last updated: Jul. 08, 2025. Accessed: Dec. 19, 2025. [Online]. Available: <https://soax.com/research/mobile-website-traffic>
- [2] Maria, "Mobile device website traffic statistics (2025 trends)," TekRevol Blog, Oct. 2025. Updated: Oct. 17, 2025. Accessed: Dec. 19, 2025. [Online]. Available: <https://www.tekrevol.com/blogs/mobile-device-website-traffic-statistics/>
- [3] X. Zhang, D. Hou, Z. Xiong, Y. Liu, S. Wang, and Y. Li, "EALLR: Energy-aware low-latency routing data driven model in mobile edge computing," *IEEE Trans. Consum. Electron.*, vol. 71, no. 2, pp. 6612–6626, May 2025.
- [4] H. Tan, M. Wang, J. Shen, P. Vijayakumar, S. Moh, and Q. Wu, "Blockchain-assisted conditional anonymous authentication and adaptive tree-based group key agreement for VANETs," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 2, pp. 2664–2679, Nov. 2025.
- [5] R. Dong et al., "One mutation fits all: Exploring universal library fuzzing based on exogenous mutation," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 6580–6591, Nov./Dec. 2025.
- [6] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [7] C. Sheng et al., "Network traffic fingerprinting for IIoT device identification: A survey," *IEEE Trans. Ind. Informat.*, vol. 21, no. 5, pp. 3541–3554, May 2025.
- [8] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11s, Sep. 2022, Art. no. 230.
- [9] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.
- [10] Z. Deng et al., "AI agents under threat: A survey of key security challenges and future pathways," *ACM Comput. Surv.*, vol. 57, 2025, Art. no. 182.
- [11] W. Zhou et al., "The security of using large language models - A survey with emphasis on ChatGPT," *IEEE/CAA J. Automatica Sinica*, vol. 12, no. 1, pp. 1–26, Jan. 2025.
- [12] X. Chen et al., "Android HIV: A study of repackaging malware for evading machine-learning detection," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 987–1001, Jul. 2020.
- [13] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–610.
- [14] Y. Li, J. Shen, S. Ji, S. Wen, T. Zhu, and Y. Xiang, "Collusion-resistant multi-replica data auditing with optimized metadata generation," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 1, pp. 417–431, Jan./Feb. 2026.
- [15] H. Shacham and B. Waters, "Compact proofs of retrievability," *J. Cryptol.*, vol. 26, no. 3, pp. 442–483, 2013.
- [16] M. Tian, C. Gao, L. Chen, H. Zhong, and J. Chen, "Proofs of retrievability with public verifiability from lattices," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3925–3935, Mar. 2025.
- [17] Y. Li, J. Shen, M. S. Obaidat, P. Vijayakumar, S. Selvaradjou, and K.-F. Hsiao, "Chameleon hash based collaborative time-series data integrity monitoring," *ACM Trans. Auton. Adaptive Syst.*, to be published, doi: [10.1145/3766541](https://doi.org/10.1145/3766541).
- [18] L. Deng, B. Wang, T. Wang, S. Feng, and S. Li, "Certificateless provable data possession scheme with provable security in the standard model suitable for cloud storage," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 3986–3998, Nov./Dec. 2023.
- [19] Y. Li, W. Zheng, S. Ji, and Y.-S. Su, "Privacy-preserving multi-party transaction auditing for social financial technology," *IEEE Trans. Consum. Electron.*, to be published, doi: [10.1109/TCE.2026.3656232](https://doi.org/10.1109/TCE.2026.3656232).
- [20] J. Feng, L. Wu, X. Fan, L. Huo, E. Wang, and Z. Zhang, "Ascina: Efficient proof of retrievability for industrial cloud storage systems," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 6356–6370, Jun. 2025.
- [21] H. Cui, Z. Wan, R. Gao, and H. Wang, "Outsourced privately verifiable proofs of retrievability via blockchain," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 1501–1514, Jul./Aug. 2024.

- [22] J. Shen, X. Chen, X. Huang, and Y. Xiang, "Public proofs of data replication and retrievability with user-friendly replication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 2057–2067, Jul./Aug. 2024.
- [23] Y. Li, J. Shen, S. Ji, and Y.-H. Lai, "Blockchain-based data integrity verification scheme in AIoT cloud-edge computing environment," *IEEE Trans. Eng. Manag.*, vol. 71, pp. 12556–12565, Apr. 2024.
- [24] Y. Li, J. Shen, P. Vijayakumar, C.-F. Lai, A. Sivaraman, and P. K. Sharma, "Next-generation consumer electronics data auditing scheme towards cloud-edge distributed and resilient machine learning," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 2244–2256, Feb. 2024.
- [25] Y. Yin, L. Xie, Z. Jiang, F. Xiao, J. Cao, and S. Lu, "A systematic review of human activity recognition based on mobile devices: Overview, progress and trends," *IEEE Commun. Surv. Tuts.*, vol. 26, no. 2, pp. 890–929, Second Quarter 2024.
- [26] A. A. Ahmed et al., "Secure AI for 6G mobile devices: Deep learning optimization against side-channel attacks," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3951–3959, Feb. 2024.
- [27] S. A. Ullah et al., "From nodes to roads: Surveying DRL applications in MEC-enhanced terrestrial wireless networks," *IEEE Commun. Surv. Tuts.*, vol. 28, pp. 1169–1208, May 2026.
- [28] Y. Xu, H. Xu, X. Chen, H. Zhang, B. Chen, and Z. Han, "Blockchain-based AR offloading in UAV-enabled MEC networks: A trade-off between energy consumption and rendering latency," *IEEE Trans. Veh. Technol.*, vol. 74, no. 11, pp. 18065–18080, Nov. 2025.
- [29] S.-Y. Zhang, C.-C. Lin, Y.-C. Peng, and D.-J. Deng, "Deployment of 5G MEC systems with safety maintenance and cybersecurity in smart agricultural parks," *IEEE Trans. Consum. Electron.*, vol. 71, no. 3, pp. 8892–8903, Aug. 2025.
- [30] Q. Zhang, S. Qian, J. Cui, H. Zhong, F. Wang, and D. He, "Blockchain-based privacy-preserving deduplication and integrity auditing in cloud storage," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1717–1729, May 2025.
- [31] J. Li, L. Zhang, J. K. Liu, H. Qian, and Z. Dong, "Privacy-preserving public auditing protocol for low-performance end devices in cloud," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 11, pp. 2572–2583, Nov. 2016.
- [32] Y. Wang, Q. Wu, B. Qin, S. Tang, and W. Susilo, "Online/offline provable data possession," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 5, pp. 1182–1194, May 2017.
- [33] Y. Yang, Y. Chen, P. Xiong, F. Chen, and J. Chen, "Decentralized self-auditing multiple cloud storage in compressed provable data possession," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 4, pp. 3901–3915, Jul./Aug. 2025.
- [34] J. Yuan and S. Yu, "Efficient public integrity checking for cloud data sharing with multi-user modification," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 2121–2129.
- [35] Y. Wang, Y. Zhang, A. Ye, J. Shen, D. Wang, and Y. Xiang, "Anonymous and efficient (t, n)-threshold ownership transfer for cloud EMRs auditing," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1710–1723, Jan. 2025.
- [36] J. Shen, F. Guo, X. Chen, and W. Susilo, "Secure cloud auditing with efficient ownership transfer," in *Proc. 25th Eur. Symp. Res. Comput. Secur.*, 2020, pp. 611–631.
- [37] R. Rabaninejad, M. A. Attari, M. R. Asaar, and M. R. Aref, "A lightweight auditing service for shared data with secure user revocation in cloud storage," *IEEE Trans. Services Comput.*, vol. 15, no. 1, pp. 1–15, Jan./Feb. 2022.
- [38] H. Wang, "Proxy provable data possession in public clouds," *IEEE Trans. Services Comput.*, vol. 6, no. 4, pp. 551–559, Fourth Quarter 2013.
- [39] H. Wang, D. He, and S. Tang, "Identity-based proxy-oriented data uploading and remote data integrity checking in public cloud," *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 6, pp. 1165–1176, Jun. 2016.
- [40] N. Lu, M. Liu, W. Shi, X. Liu, and K.-K. R. Choo, "SG-Audit: An efficient and robust cloud auditing scheme for smart grid," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4162–4179, Jul./Aug. 2024.
- [41] Z. Zhao, Y. Wang, and X. Xie, "Advancing traffic resource scheduling with cloud-edge collaboration: A virtualized digital twin perspective," *IEEE Internet Things J.*, vol. 12, no. 19, pp. 39625–39639, Oct. 2025.
- [42] H. Sedighi, F. Wuhib, and R. H. Glitho, "Dynamic task scheduling and adaptive GPU resource allocation in the cloud," *IEEE Trans. Netw. Service Manag.*, vol. 23, pp. 1112–1127, Nov. 2026.
- [43] A. Liu, M. Gao, and J. Tang, "Multi-mode instance-intensive workflow task batch scheduling in containerized hybrid cloud," *IEEE Trans. Cloud Comput.*, vol. 12, no. 1, pp. 159–173, First Quarter 2024.
- [44] W. Cao, X. Tang, T. Deng, R. Cao, and K. Li, "Resource-aware dynamic scheduling for tasks with deadline constraints on edge computing systems," *IEEE Trans. Cloud Comput.*, vol. 13, no. 4, pp. 1119–1133, Fourth Quarter 2025.

- [45] L. Zhu, K. Huang, Y. Hu, and Y. Wang, "Two-stage learning approach for semantic-aware task scheduling in container-based clouds," *IEEE Trans. Cloud Comput.*, vol. 13, no. 1, pp. 148–165, First Quarter 2025.



Meiqin Tang is currently working toward the PhD degree with the School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou, China. Her current research interests include edge computing and data auditing.



Jian Shen received the ME and PhD degrees in computer science from Chosun University, South Korea, in 2009 and 2012, respectively. Currently, he is a professor with the School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou, China. His research interests include public cryptography, cloud computing and security, data auditing and sharing, and information security systems.



Sheng Wen received the PhD degree from Deakin University, Australia, in 2014. He is currently a senior lecturer with the Swinburne University of Technology. He has received more than three million Australia Dollars funding from both academia and industries, since 2014. He is also leading a medium size research team in cybersecurity area. He has published more than 50 high-quality papers in the last six years in the fields of information security and source identification. His research interests include social network analysis and system security.



Tianqing Zhu (Member, IEEE) received the PhD degree in computer science from Deakin University, in 2014, and is currently professor and associate dean (research) with the Faculty of Data Science, City University of Macau. Her work on privacy-preserving analytics, network security, and data mining appears in *IEEE Transactions on Knowledge and Data Engineering*, *IEEE Transactions on Dependable and Secure Computing*, and *IEEE Transactions on Information Forensics and Security*. She co-authored the books *Differential Privacy and Applications* and *Location Privacy in Mobile Applications*, and serves as associate editor for several IEEE journals.



security, funded by the Australian Research Council (ARC).

Yang Xiang (Fellow, IEEE) received the PhD degree in computer science from Deakin University, Australia. He is the dean of Digital Research & Innovation Capability Platform, Swinburne University of Technology, Australia. His research interests include cyber security, which covers network and system security, data analytics, distributed systems, and networking. In particular, he is currently leading his team developing active defense systems against large-scale distributed network attacks. He is the chief investigator of several projects in network and system